10 Things You Need to Know About BERT and the Transformer Architecture That Are Reshaping the AI Landscape

25 mins read | Author Cathal Horan | March 29th, 2021

Few areas of AI are more exciting than NLP right now. In recent years language models (LM), which can perform human-like linguistic tasks, have evolved to perform better than anyone could have expected.

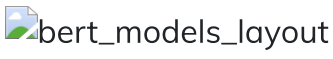
In fact, they're performing so well that people are wondering whether they're reaching a level of general intelligence, or the evaluation metrics we use to test them just can't keep up. When technology like this comes along, whether it is electricity, the railway, the internet or the iPhone, one thing is clear – you can't ignore it. It will end up impacting every part of the modern world.

It's important to learn about technologies like this, because then you can use them to your advantage. So, let's learn!

We will cover ten things to show you where this technology came from, how it was developed, how it works, and what to expect from it in the near future. The ten things are:

1. **What is BERT and the transformer, and why do I need to understand it?** Models like BERT are already massively impacting academia and business, so we'll outline some of the ways these models are used, and clarify some of the terminology around them.
2. **What did we do before these models?** To understand these models, it's important to look at the problems in this area and understand how we tackled them before models like BERT came on the scene. This way we can understand the limits of previous models and better appreciate the motivation behind the key design aspects of the Transformer architecture, which underpins most SOTA models like BERT.
3. **NLPs "ImageNet moment; pre-trained models:** Originally, we all trained our own models, or you had to fully train a model for a specific task. One of the key milestones which enabled the rapid evolution in performance was the creation of pre-trained models which could be used "off-the-shelf" and tuned to your specific task with little effort and data, in a process known as transfer learning. Understanding this is key to seeing why these models have been, and continue to perform well in a range of NLP tasks.
4. **Understanding the Transformer:** You've probably heard of BERT and GPT-3, but what about RoBERTa, ALBERT, XLNet, or the LONGFORMER, REFORMER, or T5 Transformer? The amount of new models seems overwhelming, but if you understand the Transformer architecture, you'll have a window into the internal workings of all of these models. It's the same as when you understand RDBMS technology, giving you a good handle on software like MySQL, PostgreSQL, SQL Server, or Oracle. The relational model that underpins all of the DBs is the same as the Transformer architecture that underpins our models. Understand that, and RoBERTa or XLNet becomes just the difference between using MySQL or PostgreSQL. It still takes time to learn the nuances of each model, but you have a solid foundation and you're not starting from scratch.
5. **The importance of bidirectionality**: As you're reading this, you're not strictly reading from one side to the other. You're not reading this sentence letter by letter in one direction from one side to the other. Instead, you're jumping ahead and learning context from the words and letters ahead of where you are right now. It turns out this is a critical feature of the Transformer architecture. The Transformer architecture enables models to process text in a bidirectional manner, from start to finish and from finish to start. This has been central to the limits of previous models which could only process text from start to finish.
6. **How are BERT and the Transformer Different?** BERT uses the Transformer architecture, but it's different from it in a few critical ways. With all these models it's important to understand how they're different from the Transformer, as that will define which tasks they can do well and which they'll struggle with.
7. **Tokenizers – how these models process text**: Models don't read like you and me, so we need to encode the text so that it can be processed by a deep learning algorithm. How you encode the text has a massive impact on the performance of the model and there are tradeoffs to be made in each decision here. So, when you look at another model, you can first look at the tokenizer used and already understand something about that model.
8. **Masking – smart work versus hard work**: You can work hard, or you can work smart. It's no different with Deep Learning NLP models. Hard work here is just using a vanilla Transformer approach and throwing massive amounts of data at the model so it performs better. Models like GPT-3 have an incredible number of parameters enabling it to work this way. Alternatively, you can try to tweak the training approach to "force" your model to learn more from less. This is what models like BERT try to do with masking. By understanding that approach you can again use that to look at how other models are trained. Are they employing

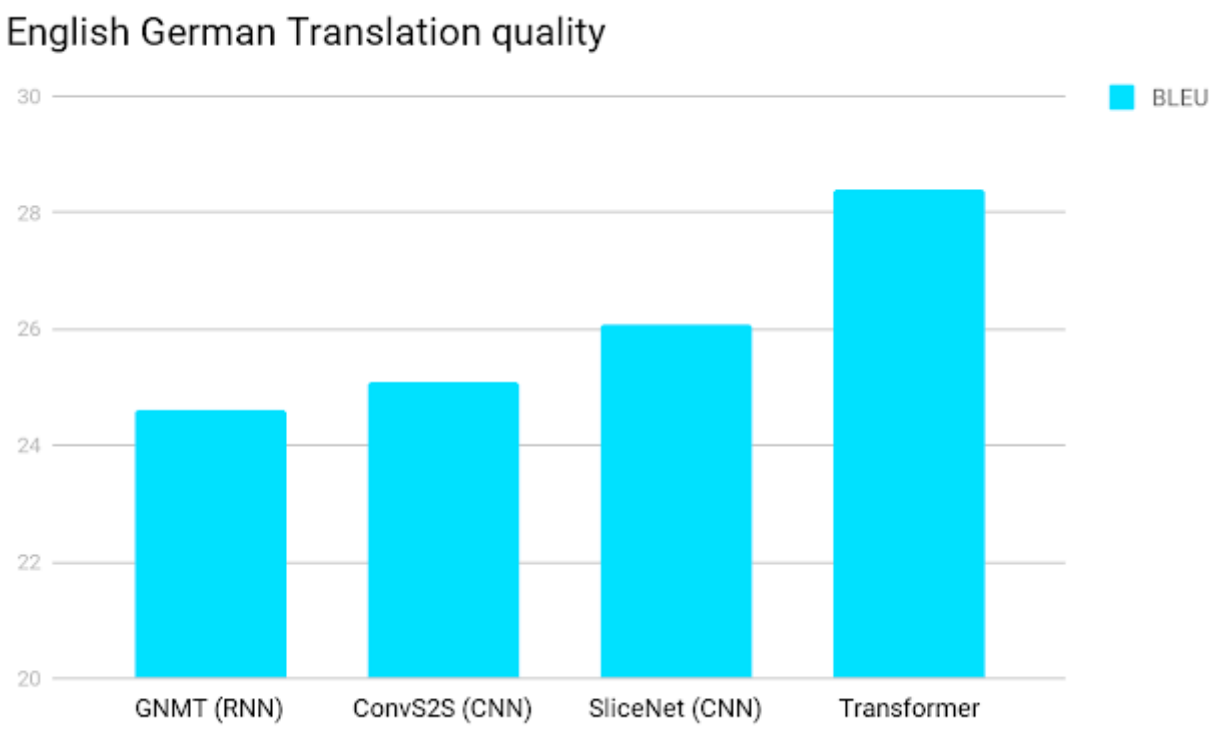
1. What is BERT and the Transformer, and why do I need to understand it?



The Transformer (Muppet) family | Source: PLM Papers

To understand the scope and speed of BERT and the Transformer, let's look at the time frame and history of the technology:

- **2017**: The Transformer Architecture was first released in December 2017 in a Google machine translation paper "*Attention Is All You Need*". That paper tried to find models that were able to translate multilingual text automatically. Prior to this, much of the machine translation techniques involved some automation, but it was supported by significant rules and linguistic based structure to ensure the translations were good enough for a service like Google Translate.
- **2018**: BERT (Bidirectional Encoder Representations from Transformers) was first released in October 2018 in "*Pre-Training of Deep Bidirectional Transformer for Language Understanding*".



Improvements in Google translate with the Transformer | Source: Google AI Blog

At first, the Transformer mainly impacted the area of machine translation. The improvements that resulted from the new approach were quickly noticed. If it had stayed in the domain of translation, then probably wouldn't be reading this article right now.

Translation is just one task in a whole range of NLP tasks that include things like tagging parts of speech (POS), recognising named entities (NER), sentiment classification, questions and answering, text generation, summarization, similarity matching and so on. Previously, each of these tasks would need a specially trained model so no one needed to learn them all, and you were generally only interested in your own domain or specific task.

READ LATER

How to Structure and Manage Natural Language Processing (NLP) Projects

This changed, however, when people started to look at the Transformer architecture and wonder if it could do something more than just translate text. They looked at the way the architecture was able to "focus attention" on specific words and process more text than other models, realising that this could be applied to a wide range of other tasks.

We'll cover the "attention" ability of the Transformer in section 5, where we show how it enabled these models to process text bidirectionally, and look at what was relevant to the specific context of the sentence being processed at that time.

Even though they have only been around for a few years these papers are already very influential and heavily cited | Source: Google Scholar most influential papers of 2020

It's at this point that the Transformer went beyond NLP. Suddenly, the future of AI was no longer about sentient robots or self-driving cars.

If these models can learn context and meaning from text and perform a wide range of linguistic tasks, does this mean they understand the text? Can they write poetry? Can they make a joke? If they're outperforming humans on certain NLP tasks, is this an example of general intelligence?

Questions like this mean that these models are no longer confined to the narrow domain of chatbots and machine translation, instead they're now part of the larger debate of AI general intelligence.

"... in a lecture published Monday, Bengio expounded upon some of his earlier themes. One of those was attention — in this context, the mechanism by which a person (or algorithm) focuses on a single element or a few elements at a time. It's central both to machine learning model architectures like Google's Transformer and to the bottleneck neuroscientific theory of consciousness, which suggests that people have limited attention resources, so information is distilled down in the brain to only its salient bits. Models with attention have already achieved state-of-the-art results in domains like natural language processing, and they could form the foundation of enterprise AI that assists employees in a range of cognitively demanding tasks". | Source: VentureBeat

Like any paradigm-shifting technology, it's important to understand if it's overhyped or undersold. Initially people thought electricity wasn't a transformative technology since it took time to re-orientate work places and urban environments to take advantage of what electricity offered.

Similarly for the railway, as well as the first days of the Internet. The point is that whether you agreed or disagreed, you needed to at least have some perspective on the rapidly developing technology at hand.

This is where we are now with models like BERT. Even if you don't use them, you still need to understand the potential impact they may have on the future of AI and, if it moves us closer to developing generally intelligent AI – on the future of society.

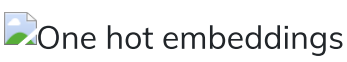
2. What existed before BERT?

BERT, and the Transformer architecture itself, can both be seen in the context of the problem they were trying to solve. Like other business and academic domains, progress in machine learning and NLP can be seen as an evolution of technologies that attempt to address failings or shortcomings of the current technology. Henry Ford made automobiles more affordable and reliable, so they became a viable alternative to horses. The telegraph improved on previous technologies by being able to communicate with people without being physically present.

Before BERT, the biggest breakthroughs in NLP were:

- **2013**: Word2Vec paper, *Efficient Estimation of Word Representations in Vector Space* was published. Continuous word embeddings started being created to more accurately identify the semantic meaning and similarity of words.

one meaning limits of Word2Vec.
Before Word2Vec, word embeddings were either simple models with massive sparse vectors which used a one hot encoding technique, or we used TF-IDF approaches to create better embeddings for ignoring common, low-information words like “*the*”, “*this*”, “*that*”.

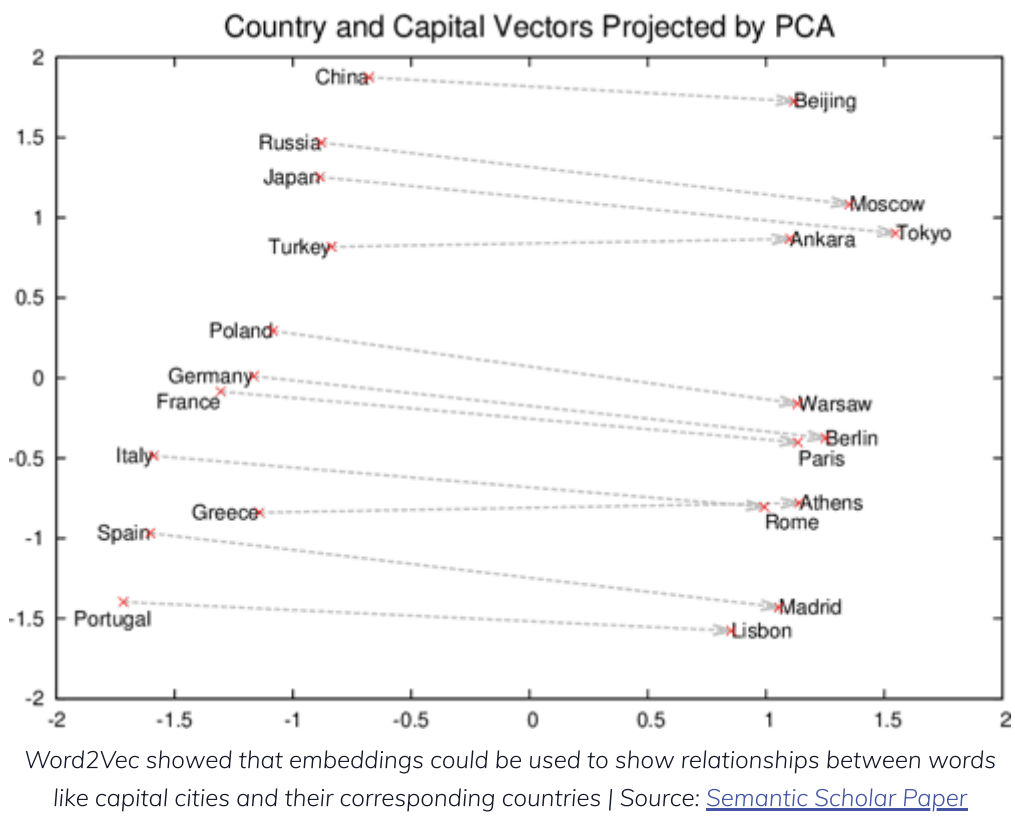


One hot embeddings are not really useful since they fail to show any relationship between words, source: FloydHub blog

These types of approaches would encode very little semantic meaning in their vectors. We could use them to classify texts and identify similarity between documents, but training them was difficult, and their overall accuracy was limited.

Word2Vec changed all that by designing two new neural network architectures; the Skip-Gram and Continuous Bag-Of-Words (CBOW), which let us train enabled word embeddings on much larger amounts of text. These approaches force neural networks to try and predict the correct words given some examples of other words in the sentence.

The theory behind this approach was that common words would be used together. For example, if you’re talking about “*phones*”, then it’s likely we will also see words like “*mobiles*”, “*iPhone*”, “*Android*”, “*battery*”, “*touch screen*” and so on. These words will likely co-occur together on a frequent enough basis that the model can begin to design a large vector with weights, which will help it predict what’s likely to occur when it sees “*phone*” or “*mobile*” and so on. We can then use these weights, or embeddings, to identify words that are similar to each other.

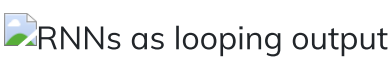


Word2Vec showed that embeddings could be used to show relationships between words like capital cities and their corresponding countries | Source: Semantic Scholar Paper

This approach was expanded with examples of text generation by doing something similar on larger sequences of text, such as sentences. This was known as a sequence to sequence approach. It expanded the scope of deep learning architectures to perform more and more complex NLP tasks.

The key problem addressed here was that language is a continuous stream of words. There’s no standard length to a sentence, and each sentence varies. Generally, these deep learning models need to know the fixed length of sequences of data they’re processing. But, that’s just not possible with text.

So, the sequence to sequence model used a technique called Recurrent Neural Networks (RNNs). With it, these architectures could perform “looping”, and process text continuously. This enabled them to create LMs which produced text in response to input prompts.

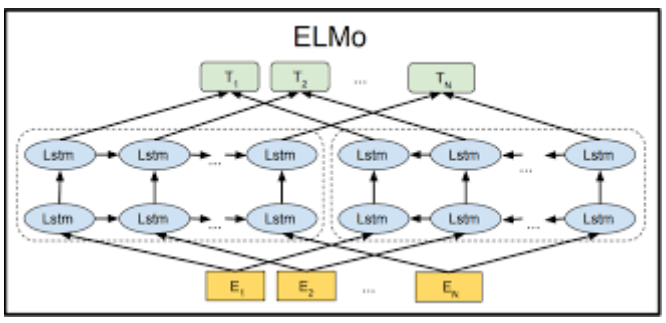


An example of how to think of RNNs as “looping” output from one part of the network to the input of the next step in a continuous

trained on.

As we noted, each new model can be seen as an attempt to improve on what has gone before. ELMo, trying to address the shortcomings of Word2Vec's static word embeddings, employed the RNN approach to train the model to recognize the dynamic nature of word meanings.

It does this by trying to dynamically assign a vector to a word, based on the sentence within which it's contained. Instead of a look-up table with a word and an embedding like Word2Vec, the ELMo model lets users input text into the model, and it generates an embedding based on that sentence. Thus, it can generate different meanings for a word depending on the context.



ELMo uses two separate networks to try and process text "bidirectionally" | Source Google AI Blog

The other important point to note here is that ELMo was the first model to try and process text non-sequentially. Previous models like Word2Vec read a word at a time, and processed each word in sequence. ELMo, to try and replicate how humans read text, processed the text in two ways:

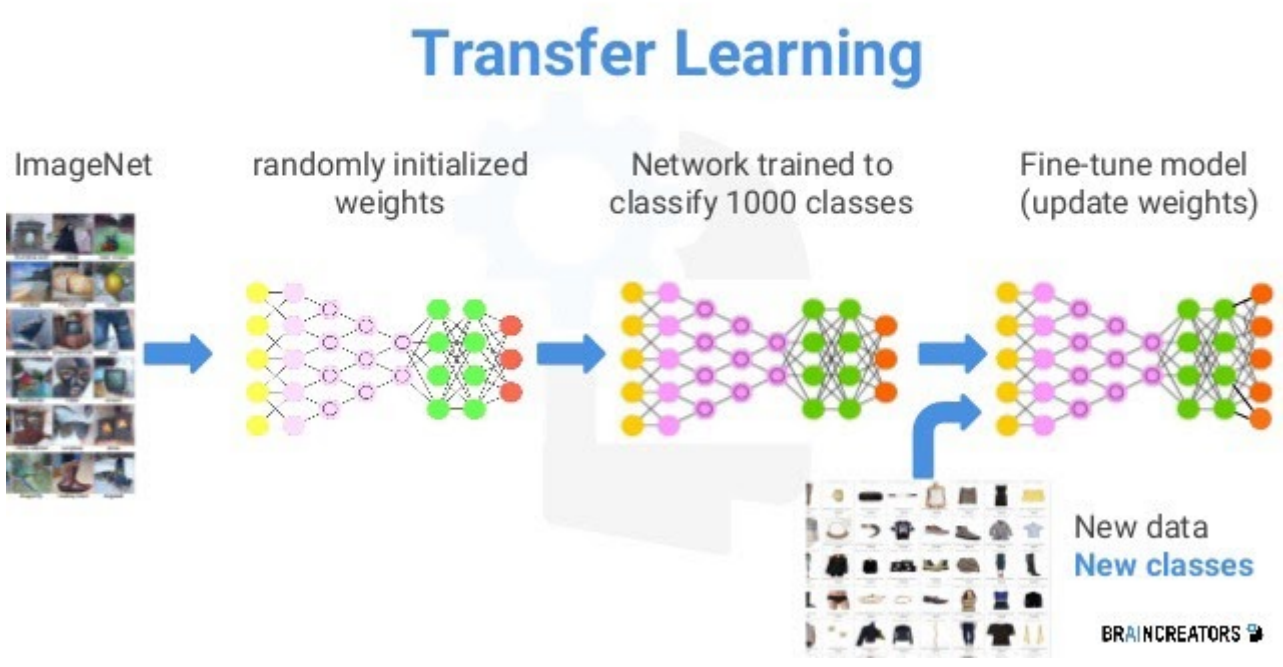
1. **Start to end**: One part of the architecture read the text as normal, from start to finish.
2. **In reverse, end to start**: Another part of the architecture read the text from back to front. The ideal being that the model might learn something else by reading "ahead".
3. **Combine**: After the text was read, both embeddings were concatenated to "combine" the meanings.

This was an attempt to read text in a bidirectional manner. While it's not "truly" bidirectional (it's more of a reversed unidirectional approach), it can be described as "shallowly" bidirectional.

In a short time frame, we'd already seen some rapid advances from Word2Vec, to more complex neural network architectures like RNNs for text generation, to context based word embeddings via ELMo. However, there were still issues with the limits of training large amounts of data using these approaches. This was a serious obstacle to the potential of these models to improve their ability to perform well on a range of NLP tasks. This is where the concept of pre-training set the scene for the arrival of models like BERT to accelerate the evolution.

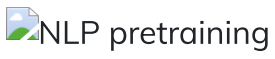
3. Pre-Trained Models

Simply put, the success of BERT (or any of the other Transformer based models) wouldn't be possible without the advent of pre-trained models. The ideal of pre-trained models is not new in deep learning. It's been practised for many years in image recognition.



These general trained models could be downloaded, and used to train on your own, much smaller dataset. Let's say you wanted to train it to recognize the faces of people in your company. You don't need to start from scratch and get the model to understand everything about image recognition, instead just build on the generally trained models and tune them to your data.

The problem with models like Word2Vec was that, while they were trained on a lot of data, they weren't general language models. You would either train Word2Vec from scratch on your data, or you would simply use Word2Vec as the first layer on your network to initialize your parameters, and then add on layers to train the model for your specific task. So, you'd use Word2Vec to process your inputs, and then design your own model layers for your sentiment classification or POS or NER task. The main limit here is that everyone is training their own models, and few people have the resources, either in terms of data or cost of computing, to train any significantly large models.



From the paper Universal Language Model Fine-tuning for Text Classification, which was one of the first LMs to bring pre-training to NLP

That was until models like ELMo arrived in 2018, as we noted in section 2. Around that time we saw other models, such as ULMFit and Open AIs first transformer model, also create pre-trained models.

This is what leading NLP researcher Sebastian Ruder called the NLPs ImageNet moment – the point where NLP researchers started to build on powerful foundations of pre-trained models to create new and more powerful NLP applications. They didn't need large amounts of money or data to do it, and these models could be used "out-of-the-box".

The reason this was critical for models like BERT is two-fold:

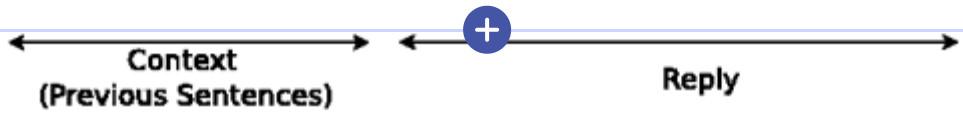
1. **Dataset Size**: Language is messy, complex, and much harder for computers to learn than identifying images. They need much more data to get better at recognising patterns in language and identifying relationships between words and phrases. Models like the latest GPT-3 were trained on 45TB of data and contain 175 billion parameters. These are huge numbers, so very few people or even organizations have the resources to train these types of models. If everyone had to train their own BERT, we would see very little progress without researchers building on the power of these models. Progress would be slow and limited to a few big players.
2. **Fine-tuning:** Pre-trained models had the dual benefit that they could be used "off-the-shelf", i.e. without any changes, a business could just plug BERT into their pipeline and use it with a chatbot or some other application. But it also meant these models could be fine-tuned to specific tasks without much data or model tweaking. For BERT, all you need is a few thousand examples and you can fine tune it to your data. Pre-training has even enabled models like GPT-3 to be trained on so much data that they can employ a technique known as zero or few shot learning. It means they only need to be shown a handful of examples to learn to perform a new task, like writing a computer program.

With the rise of pre-trained models and the advance in training and architectures from Word2Vec to ELMo, the stage was now set for BERT to arrive on the scene. At this point, we knew that we needed a way to process more data and learn more context from that data, and then make it available in a pre-trained model for others to use in their own domain specific applications.

4. The Transformer architecture

If you take one thing away from this post, make it a general understanding of the Transformer architecture and how it relates to models like BERT and GPT-3. This will let you look at different Transformer models, understand what tweaks they made to the vanilla architecture, and see what problem or task they're trying to address. This provides a key insight into what task or domain it might be better suited to.

As we learned, the original Transformer paper was called "Attention is all you need". The name itself is important since it points to how it deviates from previous approaches. In section 2, we noted the models like ELMo employed RNNs to process text sequentially in a loop-like approach.



RNNs with sequence to sequence approaches processed text sequentially until they reached an end of sentence token (<eos>). In this example an request, "ABC" is mapped to a reply "WXYZ". When the model receives the <eos> token the hidden state of the model stores the entire context of the preceding text sequence. Source: A Neural Conversation Model

Now think of a simple sentence, like "*The cat ran away when the dog chased it down the street*". For a person, this is an easy sentence to comprehend, but there's actually a number of difficulties if you think of processing this sequentially. Once you get to the "*it*" part, how do you know what it refers to? You could have to store some state to identify that the key protagonist in this sentence is the "*cat*". Then, you'd have to find some way to relate the "*it*" to the "*cat*" as you continue to read the sentence.

Now imagine that the sentence could be any number of words in length, and try to think about how you would keep track of what's being referred to as you process more and more text.

This is the problem sequential models ran into.

They were limited. They could only prioritise the importance of words that were most recently processed. As they continued to move along the sentence, the importance or relevance of previous words started to diminish.

Think of it like adding information to a list as you process each new word. The more words you process, the more difficult it is to refer to words at the start of the list. Essentially, you need to move back, one element at a time, word by word until you get to the earlier words and then see if those entities are related.

Does the "*it*" refer to the "*cat*"? This is known as the "Vanishing Gradient" problem, and ELMo used special networks known as Long Short-Term Memory Networks (LSTMs) to alleviate the consequences of this phenomenon. LSTMs did address this issue, but they didn't eliminate it.

Ultimately, they couldn't create an efficient way to "focus" on the important word in each sentence. This is the problem the Transformer network addressed by using the mechanism we already know as "attention".

This gif is from a great blog post about understanding attention in Transformers. The green vectors at the bottom represent the encoded inputs, i.e. the input text encoded into a vector. The dark green vector at the top represents the output for input 1. This process is repeated for each input to generate an output vector which has attention weights for the "importance" of each word in the input which are relevant to the current word being processed. It does this via a series of multiplication operations between the Key, Value and Query matrices which are derived from the inputs. Source: Illustrated Self-Attention.

The "Attention is all you need" paper used attention to improve the performance of machine translation. They created a model with two main parts:

1. **Encoder**: This part of the "Attention is all you need" model processes the input text, looks for important parts, and creates an embedding for each word based on relevance to other words in the sentence.
2. **Decoder**: This takes the output of the encoder, which is an embedding, and then turns that embedding back into a text output, i.e. the translated version of the input text.

The key part of the paper is not, however, the encoder or the decoder, but the layers used to create them. Specifically, neither the encoder nor the decoder used any recurrence or looping, like traditional RNNs. Instead, they used layers of "attention" through which the information passes linearly. It didn't loop over the input multiple times – instead, the Transformer passes the input through multiple attention layers.

This looks scary, and in truth it is a little overwhelming to understand how this works initially. So don't worry about understanding it all right now. The main takeaway here is that instead of looping the Transformer uses scaled dot-product attention mechanisms multiple times in parallel, i.e. it adds more attention mechanisms and then processes input in each in parallel. This is similar to looping over a layer multiple times in an RNN.
Source: Another great post on attention

The Transformer can address this by simply adding more "attention heads", or layers. Since there's no looping, it doesn't hit the vanishing gradient problem. The Transformer does still have issues with processing longer text, but it's different from the RNN problem and not something we need to get into here. For comparison, the largest BERT model consists of 24 attention layers. GPT-2 has 12 attention layers and GPT-3 has 96 attention layers.

We won't get into the fine details of how attention works here. We can look at it in another post. In the meantime, you can checkout the blog posts linked in the above diagrams. They're excellent resources on attention and how it works. The important point in this post is to understand that attention is how the Transformer architecture eliminates many of the issues we encounter when we use RNNs for NLP. The other issue we noted earlier, when looking at the limits of RNNs, was the ability to process text in a non-sequential manner. The Transformer, via the attention mechanism, enables these models to do precisely that and process text bidirectionally.

5. The importance of bidirectionality

We noted earlier that RNNs were the architectures used to process text prior to the Transformer. RNNs use recurrence or looping to be able to process sequences of textual input. Processing text in this way creates two problems:

1. **Its slow**: Processing text sequentially, in one direction, is costly since it creates a bottleneck. It's like a single lane road during peak time where there are long tailbacks, versus the road with few cars on it off peak. We know that generally speaking, these models perform better if they're trained on more data, so this bottleneck was a big problem if we wanted better models.
2. **It misses key information**: We know that humans don't read text in an absolutely pure sequential manner. As psychologist Daniel Willingham notes in his book "The Reading Mind", "*we don't read letter by letter, we read in letter clumps, figuring out a few letters at a time*". The reason is we need to know a little about what is ahead to understand what we're reading now. The same is true for NLP language models. Processing text in one direction limits their ability to learn from data

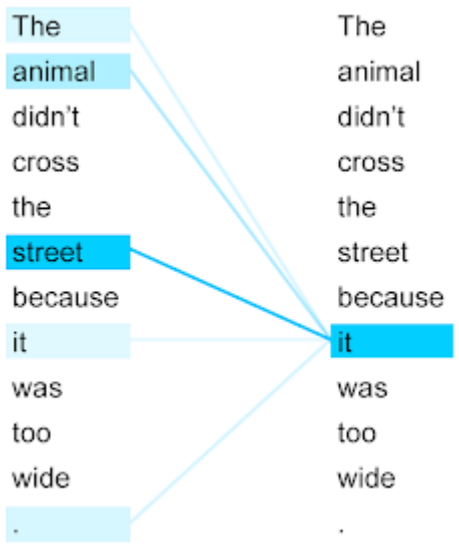
This text went viral in 2003 to show that we can read text when it is out of order. While there is some controversy around this, see here for more detail, it still shows that we do not read text strictly in a letter by letter format

We saw that ELMo attempted to address this via a method we referred to as "shallow" bidirectionality. It processed the text in one direction, then reversed the text, i.e. started from the end, and processed the text in that way. By concatenating both these embeddings, the hope was that this would help capture the different meaning in sentences like:

1. The *mouse* was on the table near the *laptop*
2. The *mouse* was on the table near the *cat*

Without understanding the nuance involved in how this process works, we can see that the ELMo approach is not ideal.

We would prefer a mechanism which enables the model to look at the other words in the sentence during the encoding, so it can know whether or not we should be worried that there is a mouse on the table! And this is exactly what the attention mechanism of the Transformer architecture enables these models to do.



This example from the Google blog shows how the attention mechanism in the Transformer network can "focus" attention on what "it" refers to, in this case the street, while also recognising the importance of the word "animal", i.e. the animal did not cross it, the street, since it was too wide.

Being able to read text bidirectionally is one of the key reasons that Transformer models like BERT can achieve such impressive results in traditional NLP tasks. As we see from the above example, being able to know what "it" refers to is difficult when you read text in only one direction, and have to store all the state sequentially.

I guess it's no surprise that this is a key feature of BERT, since the B in BERT stands for "Bidirectional". The attention mechanism of the Transformer architecture allows models like BERT to process text bidirectionally by:

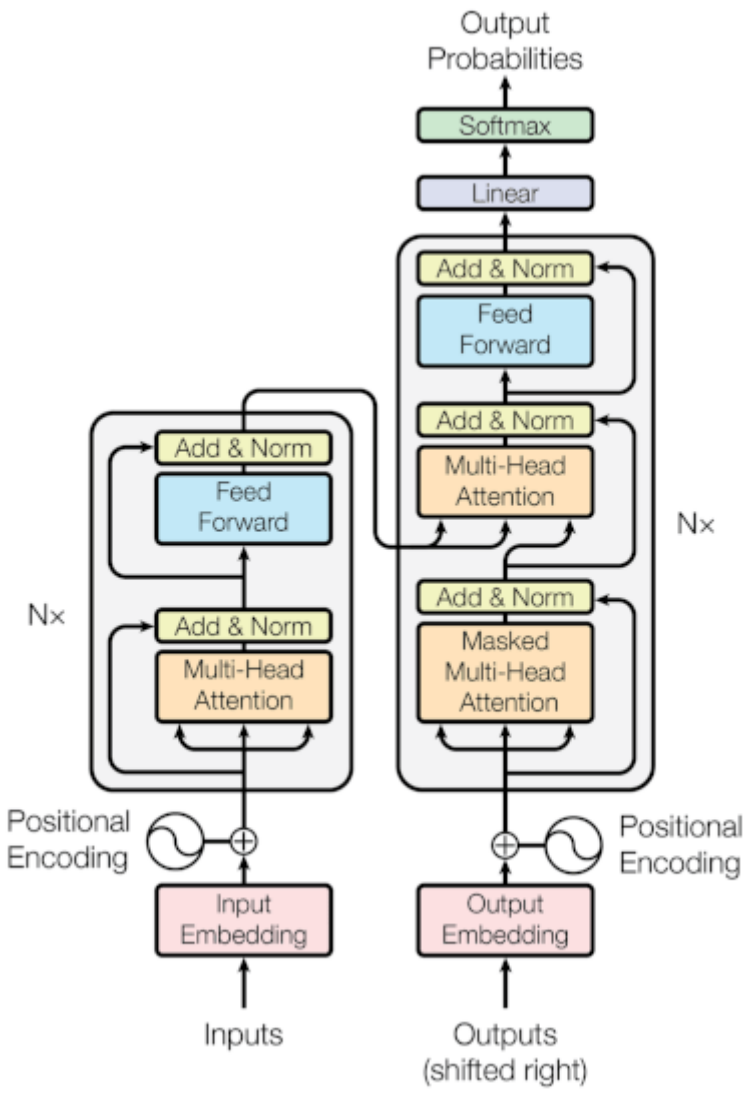
1. **Allowing parallel processing**: Transformer-based models can process text in parallel, so they're not limited by the bottleneck of having to process text sequentially like RNN-based models. This means that at any time the model is able to look at any word in the sentence it's processing. But this introduces other problems. If you're processing all the text in parallel, how do you know the order of the words in the original text? This is vital. If we don't know the order, we'll just have a bag of words-type model, unable to fully extract the meaning and context from the sentence.
2. **Storing the position of the input**: To address ordering issues, the Transformer architecture encodes the position of the word directly into the embedding. This is a "marker" that lets attention layers in the model identify where the word or text sequence they're looking at was located. This nifty little trick means that these models can keep processing sequences of text in parallel, in large volumes with different lengths, and still know exactly what order they occur in the sentence.
3. **Making lookup easy**: We noted earlier that one of the issues with RNN type models is that when they need to process text sequentially, it makes retrieving earlier words difficult. So in our "mouse" example sentence, an RNN would like to understand the relevance of the last word in the sentence, i.e. "laptop" or "cat", and how it relates to the earlier part of the sentence. To do this, it has to go from N-1 word, to N-2, to N-3 and so on, until it reaches the start of the sentence. This makes lookup difficult, and that's why context is tricky for unidirectional models to discover. By contrast, the Transformer based models can simply look up any word in the sentence at any time. In this way, it has a "view" of all the words in the sequence at every step in the attention layer. So it can "look ahead" to the end of the sentence when processing the early part of the sentence, or vice versa. (There is some nuance to this depending on the way the attention layers are implemented, e.g. encoders can look at the location of any word while decoders are limited to only looking "back" at words they have already processed. But we don't need to worry about that for now).

As a result of these factors, being able to process text in parallel, embedding the position of the input in the embedding and enabling easy lookup of each input, models like BERT can "read" text bidirectionally.

Technically it's not bidirectional, since these models are really looking at all the text at once, so it's non-directional. But it's better to understand it as a way to try and process text bidirectionally to improve the model's ability to learn from the input.

Being able to process text this way introduces some problems that BERT needed to address with a clever technique called "masking", which we'll discuss in section 8. But, now that we understand a little bit more about the Transformer architecture, we can look at the differences between BERT and the vanilla Transformer architecture in the original "Attention is all you need" paper.

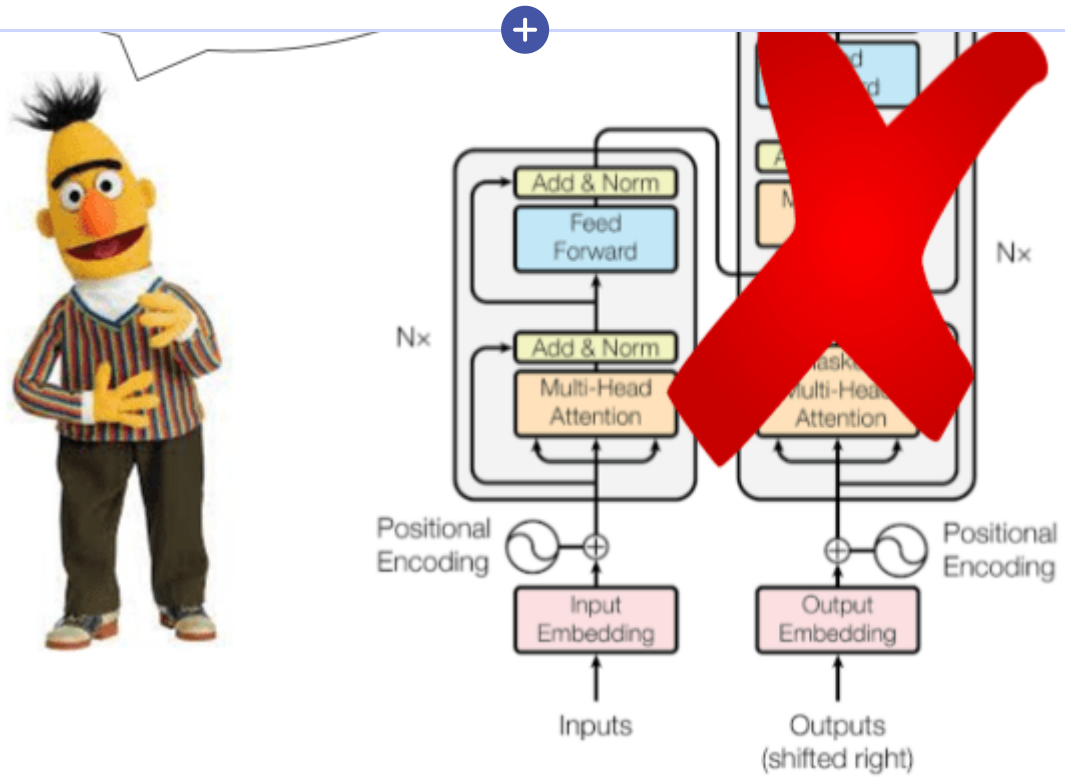
ways.



The Transformer network as described in the "Attention is all you need" paper. Notice that it has both and encoder, on the left, and decoder, on the right, which make us the network. | Source: Attention is all you need

Understanding these differences will help you know which model to use for your own unique use case. The key to understanding the different models is knowing how and why they deviate from the original Transformer architecture. In general, the main things to look out for are:

1. **Is the encoder used?** The original Transformer architecture needed to translate text so it used the attention mechanism in two separate ways. One was to encode the source language, and the other was to decode the encoded embedding back into the destination language. When looking at a new model, check if it uses the encoder. This means it's concerned with using the output in some way to perform another task, i.e. as an input to another layer for training a classifier, or something of that nature.
2. **Is the decoder used?** Alternatively, a model might not use the encoder part, and only use the decoder. The decoder implements the attention mechanism slightly differently to the encoder. It works more like a traditional language model, and only looks at previous words when processing the text. This would be suitable for tasks like language generation, which is why the GPT models use the decoder part of the Transformer, since they're mainly concerned with generating text in response to an input sequence of text.
3. **What new training layers are added?** The last thing to look at is what extra layers, if any, the model adds to perform training. The attention mechanism opens up a whole range of possibilities by being able to process text in parallel and bidirectionally, as we mentioned earlier. Different layers can build on this, and train the model for different tasks like question and answering, or text summarization.



BERT only uses the encoder part of the original Transformer network

Now that we know what to look for, how does BERT differ from the vanilla Transformer?

1. **BERT uses the encoder**: BERT uses the encoder part of the Transformer, since it's goal is to create a model that performs a number of different NLP tasks. As a result, using the encoder enables BERT to encode the semantic and syntactic information in the embedding, which is needed for a wide range of tasks. This already tells us a lot about BERT. First, it's not designed for tasks like text generation or translations, because it uses the encoder. It can be trained on multiple languages, but it's not a machine translation model itself. Similarly, it can still predict words, so it can be used as a text generating model, but that's not what it's optimized for.
2. **BERT doesn't use the decoder**: As noted, BERT doesn't use the decoder part of the vanilla Transformer architecture. So, the output of BERT is an embedding, not a textual output. This is important – if the output is an embedding, it means that whatever you use BERT for you'll need to do something with the embedding. You can use techniques like cosine similarity to compare embeddings and return a similarity score for example. By contrast, if you used the decoder, the output would be a text so you could use that directly without needing to perform any further actions.
3. **BERT uses an innovative training layer:** BERT takes the output of the encoder, and uses that with training layers which perform two innovative training techniques, masking and Next Sentence Prediction (NSP). These are ways to unlock the information contained in the BERT embeddings to get the models to learn more information from the input. We will discuss these techniques more in section 8, but the gist is that BERT gets the Transformer encoder to try and predict hidden or masked words. By doing this, it forces the encoder to try and "learn" more about the surrounding text and be better able to predict the hidden or "masked" word. Then, for the second training technique, it gets the encoder to predict an entire sentence given the preceding sentence. BERT introduced these "tweaks" to take advantage of the Transformer, specifically the attention mechanism, and create a model that generated SOTA results for a range of NLP tasks. At the time, it surpassed anything that had been done before.

Now that we know how BERT differs from the vanilla Transformer architecture, we can look closely at these parts of the BERT model. But first, we need to understand how BERT "reads" text.

7. Tokenizers: How BERT reads

When we think about models like BERT, we often overlook one important part of the process: how do these models "read" the input to be able to learn from the vast amounts of text they're trained on?

You might think this is the easy part. Just process each word at a time, identify each word by a space separating them, and then pass that to the attention layers to do their magic.

Tokenization seems straightforward, it just breaks the sentence up into words. But it turns out this is not as easy as it seems. | Source: FloydHub

However, a few problems arise when we try and tokenize text via words or other simple methods such as punctuation, for example:

1. **Some languages don't separate words via spaces**: Using a word-level approach means the model couldn't be used in languages such as Chinese, where word separation isn't a trivial task.
2. **You would need a massive vocabulary**: If we split things by words, we'd need to have a corresponding embedding for every possible word we might encounter. That's a big number. And how do you know you saw every possible word in your training dataset? If you didn't, the model won't be able to process a new word. This is what happened in the past, and models were forced to the <UNK> token to identify when it had encountered an unknown word.
3. **Larger vocabularies slow your model down**: Remember we want to process more data to get our model to learn more about language and perform better on our NLP tasks. This is one of the main benefits of the Transformer – we can process much more text than any previous model, which helps make these models better. However, if we use word level tokens, we need a large vocabulary which adds to the size of the model and limits its ability to be trained on more text.

HuggingFace has a great tokenizer library. It includes great example tutorials on how the different approaches work. Here, for example, it shows how the base vocabulary is created based on the frequency of words in the training data. This shows how a word like "hug" ends up being tokenized as "hug", while "pug" is tokenized by two subword parts, "p" and "ug".

How did BERT solve these problems? It implemented a new approach to tokenization, WordPiece, which applied a sub-word approach to tokenization. WordPiece solves many of the problems previously associated with tokenization by:

1. **Using sub-words instead of words**: Instead of looking at whole words, WordPiece breaks words down into smaller parts, or building blocks of words, so that it can use them to build up different words. For example, think of a word like "learning". You could break that into three parts, "lea", "rn" and "ing". This way, you can make a wide range of words using the building blocks:
    - Learning = lea + rn + ing
    - Learn = lea + rn

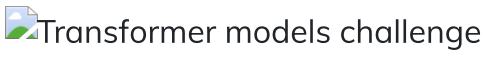
You can then combine these together with other subword units to make other words:

- Burn = bu + rn
- Churn = chu +rn
- Turning = tur + rrn + ing
- Turns = tu + rn + s

You can create whole words for your most common terms, but then create subword for the other words that don't appear as often. Then you can be confident that you'll tokenize any word by using the building blocks you've created. If you encounter a completely new word, you can always piece it together character by character, e.g. LOL = l + o + l, since the subword token library also includes each single character. You can build up any word even if you've never seen it. But generally, you should have some subword unit you can use and add on a few characters as needed.

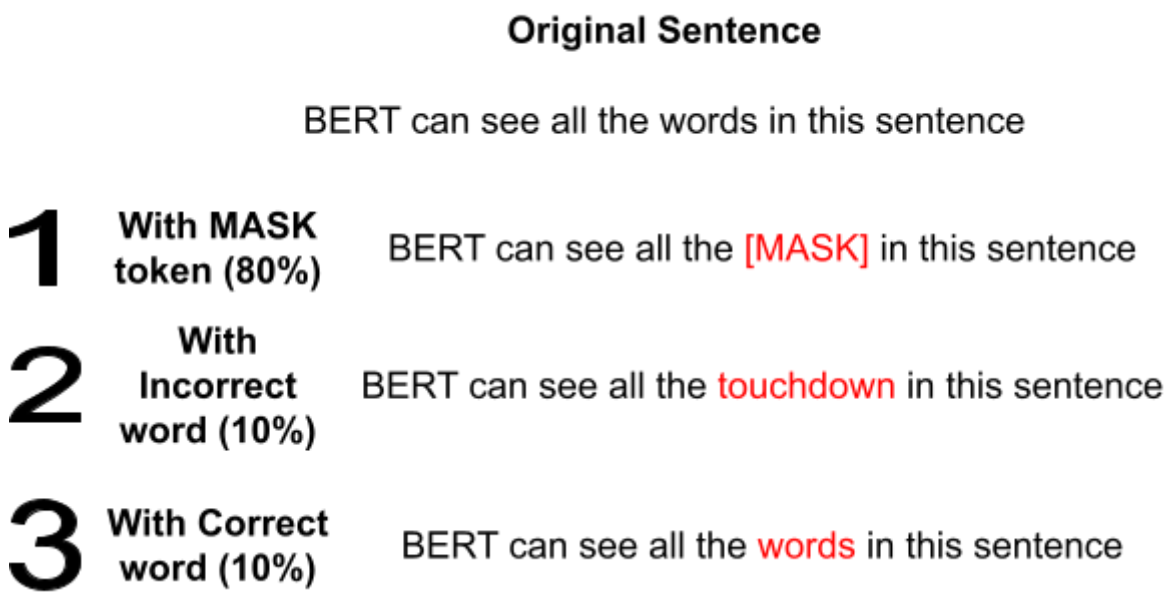
2. **Creating a small vocabulary size**: Since we don't need a token for each word, we can instead create a relatively small vocabulary. BERT for example uses about 30,000 tokens in its library. That may seem large, but think about every possible word ever invented and used, and all the different ways they can be used. To cover that, you'd need vocabularies with millions of tokens, and you still wouldn't cover everything. Being able to do this with 30,000 tokens is incredible. It also means we don't need to use the <UNK> token to still have a small model size and train on large amounts of data.
3. **It still assumes words are separated by spaces, but ...** While WordPiece does assume that words are separated by spaces, new subword token libraries such as SentecePiece (used with the Transformer based model) or the Multilingual Universal

As we noted in section 6, when you're looking at different Transformer based models, it can be interesting to look at how they differ in their approach to training. In the case of BERT, the training approach is one of the most innovative aspects. Transformer models offer enough improvements just with the vanilla architecture that you can just train them using the traditional language model approach and see massive benefits.



One problem with BERT and other Transformer based models which use the encoder is that they have access to all the words at input time. So asking it to predict the next words is too easy. It can "cheat" and just look it up. This was not an issue with RNNs based models which could only see the current word and not the next one, so they couldn't "cheat" this way. Source: Stanford NLP

This is where you predict future tokens from previous tokens. The previous RNNs used autoregressive techniques to train their models. GPT models similarly use an autoregressive approach to training their models. Except with the Transformer architecture, (i.e. the decoder as we noted earlier), they can train on more data than ever before. And, as we now know, the models can learn context better via the attention mechanism and the ability to process input bidirectionally.



BERT uses a technique called masking to prevent the model from "cheating" and looking ahead at the words it needs to predict. Now it never knows whether the word it's actually looking at is the real word or not, so it forces it to learn the context of all the words in the input, not just the words being predicted.

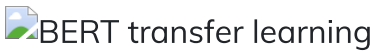
Instead, BERT used an innovative technique to try and "force" the models to learn more from given data. This technique also raises a number of interesting aspects about how deep learning models interact with a given training technique:

1. **Why is masking needed?** Remember that the Transformer allows models to process text bidirectionally. Essentially, the model can see all the words in the input sequence at the same time. Previously, with RNN models, they only saw the current work in the input and had no idea what was the next word. Getting those models to predict the next word was easy, the model didn't know it so had to try and predict it and learn from that. However, with the Transformer, BERT can "cheat" and look at the next word so it will learn nothing. It's like taking a test and being given the answers. You won't study if you know the answer will always be there. BERT uses masking to solve this.
2. **What is masking?** Masking (also known as a cloze test) simply means that instead of predicting the next word, we hide or "mask" a word, and then force the model to predict that word. For BERT, 15% of the input tokens are masked before the model gets to see them, so there's no way for it to cheat. To do this, a word is randomly selected and simply replaced with a "[MASK]" token, and then fed into the model.
3. **No labels needed**: The other things to always remember for these tasks is that if you are designing a new training technique, it's best if you don't need to label or manually structure the data for training. Masking achieves this by requiring a simple approach to enable training on vast amounts of unstructured data. This means it can be trained in a fully unsupervised manner.

learn context for the non-masked words, we need to replace some of the tokens with a random word, and some with the correct word. This means BERT can never know if the unmasked word it's being asked to predict is the correct word. If, as an alternative approach, we used the MASK token 90% of the time and then used an incorrect word 10% of the time, BERT would know while predicting a non-masked token that it's always the wrong word. Similarly, if we used only the correct word 10% of the time, BERT would know it's always right, so it would just keep reusing the static word embedding it learned for that word. It would never learn a contextual embedding for that word. This is a fascinating insight into how Transformer models represent these states internally. No doubt we'll see more training tweaks like this pop up in other models.

9. Fine tune and transfer learning

As should be clear by now, BERT broke the mold in more ways than one when it was published a few years ago. One other difference was the ability to be tuned to a specific domain. This builds on many of the things we've already discussed, such as being a pre-trained model that meant people didn't need access to a large dataset to train it from scratch. You can build on models that have learned from a much larger dataset, and "transfer" that knowledge to your model for a specific task or domain.



In Transfer Learning we take knowledge learned in one setting, usually via a very large dataset, and apply it to another domain where, generally, we have much less data available. | Source *Sebastian Ruder blog post*

The fact that BERT is a pre-trained model means we can fine tune it to our domain, because:

1. **BERT can perform transfer learning**: Transfer learning is a power concept which was first implemented for machine vision. Models trained on ImageNet were then available for other "downstream" tasks, where they could build on the knowledge of the model trained on more data. In other words, these pre-trained models can "transfer" the knowledge they learned on the large dataset to another model, which needs much less data to perform well at a specific task. For machine vision, the pre-trained models know how to identify general aspects of images, such as lines, edges, the outlines of faces, the different objects in a picture and so on. They don't know fine-grained detail like individual facial differences. A small model can be easily trained to transfer this knowledge to its tasks, and recognize faces or objects specific to its task. If you wanted to identify a diseased plant, you didn't need to start from scratch.
2. **You can choose relevant layers to tune**: Although it's still a matter of ongoing research, it appears that higher layers of the BERT models learn more contextual or semantic knowledge while lower levels tend to perform better on syntactic related tasks. The higher layers are generally more related to task specific knowledge. For fine-tuning, you can add your own layer on top of BERT, and use a small amount of data to train that on some task, like classification. In those cases, you'd freeze the parameters of the later layer, and only allow your added layer parameters to change. Alternatively, you can "unfreeze" these higher layers and fine-tune BERT by letting these values change.
3. **BERT needs much less data**: Since it appears that BERT has already learned some "general" knowledge about language, you need much less data to fine-tune it. This means you can use labelled data to train a classifier, but you need to label much less data, or you can use the original training technique of BERT, like NSP, to train it on unlabelled data. If you have 3000 sentence pairs, you can fine-tune BERT with your unlabelled data.

All this means that BERT, and many other Transformer models, can be easily tuned to your business domain. While there do seem to be some issues with fine-tuning, and many models have trained BERT from scratch for unique domains like Covid19 information retrieval, it's still a big shift in the design of these models that they're capable of being trained on unsupervised and relatively small amounts of data.

Avocado chairs designed by a decoder based Transformer model which was trained with images as well as text. | Source: OpenAI

Now that we've come to the end of our whirlwind review of BERT and the Transformer architecture, let's look forward to what we can expect in the future for this exciting area of Deep Learning:

1. **Questions about limits of these models**: There is a fascinating, almost philosophical, discussion around the potential limits of trying to extract meaning from text alone. Some recently published papers show how there is a new form of thinking about what deep learning models can learn from language. Are these models doomed to hit a point of diminishing returns in terms of training on more and more data? These are exciting questions, which are moving outside of the boundaries of NLP into the broader domain of general AI. For more on the limits of these models, see our previous Neptune AI post on this topic.
2. **Interpretability of models**: It's often said that we start to use a technology, and only later on figure out how it works. For example, we learned to fly planes before we really understood everything about turbulence and aerodynamics. Similarly, we're using these models and we really don't understand what or how they're learning. For some of this work, I strongly recommend Lena Voita's blog. Her research on BERT, GPT and other Transformer models is incredible, and an example of the growing body of work where we're starting to reverse-engineer how these models are working.
3. **An image is worth a thousand words**: One interesting potential development of the Transformer models is that there's some work already underway which combines text and vision to enable models to generate images from sentence prompts. Similarly, Convolution Neural Networks (CNNs) were the core technology of machine vision until recently, when Transformer based models started being used in this field. This is an exciting development, since the combination of vision and text has the potential to improve the performance of these models rather than using text alone. As we noted earlier, what started out as a machine translation model has morphed into a technology that seems closer than any other area of AI to realise the holy grail of general intelligence. Watch this space closely, it might just be the most exciting area of AI at the moment.

Conclusion

And that's it. If you made it this far – thank you so much for reading! I hope this article helped you understand BERT and the Transformer architecture.

It's truly one of the most interesting domains in AI right now, so I encourage you to keep on exploring and learning about this. You can start with the various different links that I left throughout this article.

Good luck on your AI journey!

Cathal Horan

Works on the ML team at Intercom where he creates AI products that help businesses improve their ability to support and communicate with their customers. He is interested in the intersection of philosophy and technology and is particularly fascinated by how technologies like deep learning can create models which may one day understand human language. He recently completed an MSc in business analytics. His primary degree is in electrical and electronic engineering, but he also boasts a degree in philosophy and an MPhil in psychoanalytic studies.

Follow me on

READ NEXT

Let me share a story that I've heard too many times.

> *"... We were developing an ML model with my team, we ran a lot of experiments and got promising results...*
>
> *...unfortunately, we couldn't tell exactly what performed best because we forgot to save some model parameters and dataset versions...*
>
> *...after a few weeks, we weren't even sure what we have actually tried and we needed to re-run pretty much everything"*
>
> *– unfortunate ML researcher.*

And the truth is, when you develop ML models you will run a lot of experiments.

Those experiments may:

- use different models and model hyperparameters
- use different training or evaluation data,
- run different code (including this small change that you wanted to test quickly)
- run the same code in a different environment (not knowing which PyTorch or Tensorflow version was installed)

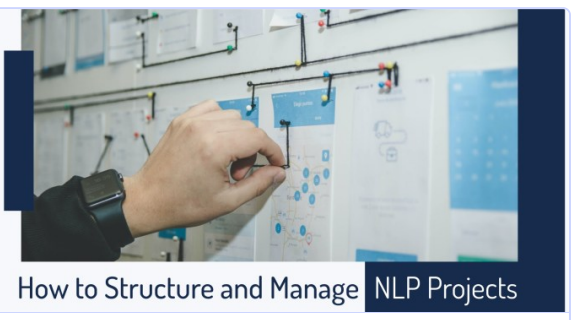
And as a result, they can produce completely different evaluation metrics.

Keeping track of all that information can very quickly become really hard. Especially if you want to organize and compare those experiments and feel confident that you know which setup produced the best result.

Have you heard of **Experiment Tracking**?

Learn **what** it is, **why** it matters, and **how** to implement it.

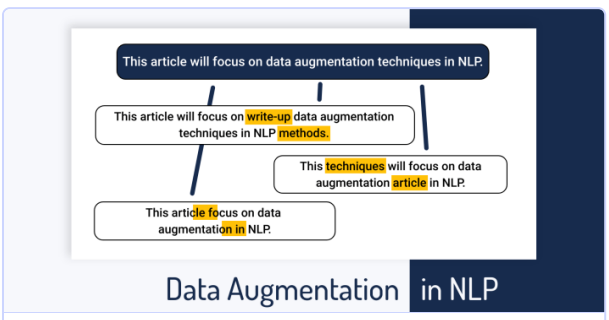
Read now



How to Structure and Manage Natural Language Processing (NLP) Projects

by Dhruvil Karani, October 12th, 2020

Read more



Data Augmentation in NLP: Best Practices From a Kaggle Master

by Shahul ES, July 20th, 2020

Read more

AI Limits: Can Deep Learning Models Like BERT Ever Understand Language?

by Cathal Horan, November 11th, 2020

Read more

Deep Dive in RNN

by Nilesh Barla, March 18th, 2021

Top ML articles from our blog in your inbox every month.

Type your email here

Get Newsletter

GDPR complaint. Privacy policy.